

Name: Tejarvi M

Reg no: 192511047

Course: DATA STRUCTURE

Date: 30/08/26

Lo5 - AT1 - PSEUDOCODE

1) Social Network - Connected components + Shortest path

DFS (G, v):

visited[v] = true

for each u in Adj[v]:

if not visited[u]:

DFS(G, u)

BFS (G, Source):

distance[source] = 0

put source in Queue

while Queue not empty:

v = remove Queue

for each u in Adj[v]:

if not visited(u):

distance[u] = distance[v] + 1;

add u to Queue

2) Task Dependencies - Topological Sort

Create directed graph G

Calculate indegree of all tasks

Add indegree = 0 task to Queue

while Queue not empty:

 v = remove Queue

 print v

 for each u in Adj[v]:

 indegree[u] --

 if indegree[u] == 0:

 add u to Queue

if all tasks not printed:

 print "Circle exists"

3) Task Scheduling System

Create directed graph G

Add tasks as vertices

Add dependency as directed edge

checks for cycles using DFS

if cycle exist:

print "invalid schedule"

else:

Topological Sort (G)

print execution order

4) DFS + cycle Detection

DFS [u]:

visited[v] = true

path[v] = true

for each u in Adj[v]:

if not visited[u]:

if DFS[u]:

return true

else:

if path[u]:

return true

path[v] = false

return false

5.) Delivery Routing - Shortest path

Create graph G

Add locations as vertices

Add roads with distance as weight

Choose source and destination

Run Dijkstra (G , source)

Print shortest route and distance.

6.) Delivery Routing - MST

Create weighted graph G

Add locations as vertices

Add roads with cost

Run Prim / Kruskal algorithm

Print MST

Print total minimum costs

7.) Kruskal's MST

Sort all edges by weight

Create empty MST

Make each vertex a separate set

for each edge (u, v) :

if $\text{Find}(u) \neq \text{Find}(v)$:

Add edge to MST

Union (u, v)

Stop when MST has $v-1$ edges

Print MST

8) Floyd - Warshall - EPS

Initialize distance matrix D .

for $k = 1$ to v :

for $i = 1$ to v :

for $j = 1$ to v :

$D[i][j] = \min[D[i][j],$

$D[i][k] + D[k][j]]$

for $i = 1$ to v :

if $D[i][i] < 0$:

print "Negative cycle"

Print D

9.) Priority Topological Sort

Calculate indegree

put all indegree = 0 nodes in priority Queue

while Queue not empty:

v = remove highest - priority node

print v

for each u in Adj[v]:

$\text{indegree}(u) = \text{indegree}(u) - 1$

if $\text{indegree}(u) == 0$:

add u to Queue

10.) Modified Dijkstra - Avoid Certain Nodes

Mark forbidden nodes

Set distance (source) = 0

while unvisited nodes exist:

choose minimum - distance allowed node v .

for each neighbor u :

if u is forbidden:

continue:

if $\text{distance}[u] > \text{distance}[v] + \text{weight}(v, u)$:

$\text{distance}[u] = \text{distance}[v] + \text{weight}(v, u)$: